

Corrigé détaillé — Sujet d'entraînement A

Singleton, Enum et Strategy

Exercice 1 — Singleton et Enum

Question 1 — GestionnaireScores (Singleton classique)

L'idée du Singleton « classique » repose sur trois ingrédients qui doivent **tous** être présents :

- un constructeur `private`, pour empêcher toute création d'instance depuis l'extérieur de la classe ;
- un champ `static` qui contient l'unique instance (initialement `null`, ou initialisé directement) ;
- une méthode `static` (souvent `getInstance()`) qui crée l'instance au premier appel puis la renvoie systématiquement.

```
import java.util.ArrayList;
import java.util.List;

public class GestionnaireScores {

    private static GestionnaireScores instance;

    private List<Score> meilleursScores;

    private GestionnaireScores() {
        this.meilleursScores = new ArrayList<Score>();
    }

    public static GestionnaireScores getInstance() {
        if (instance == null) {
            instance = new GestionnaireScores();
        }
        return instance;
    }

    public void ajouterScore(String nom, int valeur) {
        Score s = new Score(nom, valeur);
        int i = 0;
        while (i < meilleursScores.size() && meilleursScores.get(i).getValeur() >= valeur) {
            i++;
        }
        meilleursScores.add(i, s);
    }

    public List<Score> getTop(int n) {
        int limite = Math.min(n, meilleursScores.size());
        return new ArrayList<Score>(meilleursScores.subList(0, limite));
    }
}
```

Points clés :

- `ajouterScore` fait une **insertion triée** : on cherche la première position où le score à insérer est strictement supérieur au score déjà en place (condition de boucle `>= valeur`, donc on s'arrête dès qu'on trouve un score strictement plus petit), puis on insère à cette position avec `add(i, s)`. C'est plus économe qu'ajouter en fin de liste puis retrier à chaque fois.
- `getTop(n)` renvoie une **copie** (`new ArrayList<>(...)`) du sous-ensemble demandé plutôt que la sous-liste elle-même ou la liste interne directement. C'est une bonne pratique d'encapsulation : si on renvoyait directement `meilleursScores` (ou une vue obtenue par `subList`, qui reste liée à la liste d'origine), l'appelant pourrait modifier

l'état interne du Singleton de l'extérieur, ce qui casserait l'encapsulation.

- `Math.min(n, meilleursScores.size())` gère proprement le cas où la liste contient moins de `n` éléments, comme demandé.

Question 2 — Pourquoi `private` et `static` ?

Le constructeur doit être `private` car c'est le **seul moyen** d'empêcher la création d'instances depuis l'extérieur de la classe : si le constructeur était `public` ou même par défaut, n'importe quel code pourrait faire `new GestionnaireScores()` et créer une deuxième instance, ce qui viole l'unicité recherchée par le pattern.

Le champ contenant l'unique instance doit être `static` car il doit exister et être accessible **avant même qu'une instance n'existe** (au moment du tout premier appel à `getInstance()`), et il doit être **partagé par tous les appels**, indépendamment de toute instance. Un champ d'instance n'aurait pas de sens ici puisqu'il n'existe par définition aucune instance pour le porter tant que le Singleton n'a pas été créé.

Question 3 — `GestionnaireScores` (Singleton par enum)

```
import java.util.ArrayList;
import java.util.List;

public enum GestionnaireScores {

    INSTANCE;

    private List<Score> meilleursScores = new ArrayList<Score>();

    public void ajouterScore(String nom, int valeur) {
        Score s = new Score(nom, valeur);
        int i = 0;
        while (i < meilleursScores.size() && meilleursScores.get(i).getValeur() >= valeur) {
            i++;
        }
        meilleursScores.add(i, s);
    }

    public List<Score> getTop(int n) {
        int limite = Math.min(n, meilleursScores.size());
        return new ArrayList<Score>(meilleursScores.subList(0, limite));
    }
}
```

Utilisation : `GestionnaireScores.INSTANCE.ajouterScore("Alice", 120);`

Une `enum` Java garantit **par construction** qu'il n'existe qu'une seule instance de chaque constante déclarée (ici une seule constante, `INSTANCE`) : la JVM se charge elle-même de cette unicité, sans qu'on ait besoin d'écrire de constructeur privé explicite ni de champ statique nullable — la logique de `getInstance()` n'a tout simplement plus de raison d'exister.

Question 4 — Avantage de l'enum (désérialisation / réflexion)

Avec un Singleton « classique », deux attaques classiques peuvent casser l'unicité :

- **désérialisation** : si la classe implémente `Serializable`, désérialiser un objet ne repasse jamais par le constructeur — on obtient une nouvelle instance distincte de celle gérée par `getInstance()`, sauf à redéfinir manuellement la méthode `readResolve()`.
- **réflexion** : un constructeur `private` reste accessible via l'API de réflexion en appelant `setAccessible(true)` sur le `Constructor`, ce qui permet d'instancier la classe une deuxième fois malgré le `private`.

Une `enum` est **intrinsèquement protégée** contre ces deux attaques : la JVM interdit l'instanciation d'une constante d'énumération par réflexion, et la désérialisation des enums est gérée nativement par la JVM en renvoyant toujours la constante existante plutôt qu'une copie. C'est pour cette raison que le Singleton par enum est souvent considéré comme

l'implémentation la plus robuste.

Exercice 2 — Strategy

Question 1 — Interface `CritereDeTri`

```
@FunctionalInterface
public interface CritereDeTri {
    int compare(ObjetZork o1, ObjetZork o2);
}
```

Une interface est dite *fonctionnelle* si elle ne possède **qu'une seule méthode abstraite** — c'est précisément ce qui permet de l'implémenter avec une expression lambda ou une référence de méthode, en plus d'une classe anonyme classique. L'annotation `@FunctionalInterface` n'est pas obligatoire mais signale explicitement cette intention (et fait échouer la compilation si on ajoute une deuxième méthode abstraite par erreur).

Question 2 — Classe `Inventaire`

```
import java.util.ArrayList;
import java.util.List;

public class Inventaire {
    private List<ObjetZork> objets;

    public Inventaire() {
        this.objets = new ArrayList<ObjetZork>();
    }

    public void ajouter(ObjetZork o) {
        objets.add(o);
    }

    public void trier(CritereDeTri critere) {
        objets.sort((o1, o2) -> critere.compare(o1, o2));
    }

    public List<ObjetZork> getObjets() {
        return objets;
    }
}
```

Le cœur du pattern Strategy est ici : `Inventaire` ne connaît **aucun** critère de tri concret, elle reçoit en paramètre un objet `CritereDeTri` (la « stratégie ») et lui délègue entièrement la logique de comparaison. On peut ainsi changer de comportement de tri à l'exécution sans toucher au code d' `Inventaire`.

Détail technique : `List.sort` attend un `Comparator<? super ObjetZork>`. Comme `CritereDeTri` n'est pas elle-même un `Comparator`, on l'adapte avec une lambda `(o1, o2) -> critere.compare(o1, o2)` qui, elle, est bien un `Comparator<ObjetZork>` valide (méthode abstraite unique `compare(T, T)` de même signature).

Question 3 — Critères concrets en lambda

```
CritereDeTri parPoidsCroissant = (o1, o2) -> o1.getPoids() - o2.getPoids();

CritereDeTri parNomAlphabetique = (o1, o2) -> o1.getNom().compareTo(o2.getNom());
```

`o1.getPoids() - o2.getPoids()` est négatif si `o1` est plus léger que `o2`, ce qui correspond bien à un tri croissant.

`String.compareTo` renvoie déjà directement un entier dont le signe correspond à l'ordre lexicographique, donc on peut le réutiliser tel quel.

Question 4 — `TriParPoidsDecroissant` (Strategy + Singleton)

```
public class TriParPoidsDecroissant implements CritereDeTri {  
  
    private static TriParPoidsDecroissant instance;  
  
    private TriParPoidsDecroissant() {  
    }  
  
    public static TriParPoidsDecroissant getInstance() {  
        if (instance == null) {  
            instance = new TriParPoidsDecroissant();  
        }  
        return instance;  
    }  
  
    public int compare(ObjetZork o1, ObjetZork o2) {  
        return o2.getPoids() - o1.getPoids();  
    }  
}
```

On retrouve exactement la structure du Singleton classique vue à l'exercice 1 (constructeur `private`, champ `static`, méthode d'accès `static`), appliquée ici à une classe qui, en plus, implémente une interface de stratégie. C'est une situation courante en pratique : une stratégie « sans état » (qui ne dépend d'aucune donnée propre à une instance particulière) n'a aucune raison d'être instanciée plusieurs fois, donc autant garantir son unicité.

Pour le tri décroissant, on inverse simplement l'ordre de la soustraction par rapport au critère croissant : `o2.getPoids() - o1.getPoids()`.

Pièges fréquents

1. **Oublier que `getTop` / les accesseurs d'un Singleton doivent renvoyer une copie** quand ils exposent une structure mutable interne, sous peine de casser l'encapsulation que le pattern est censé garantir.
2. **Confondre `CritereDeTri` et `Comparator`** : ce sont deux interfaces différentes même si elles ont la même forme (un seul `compare(T, T)`) — il faut explicitement adapter l'une à l'autre via une lambda quand on appelle `List.sort`.
3. **Implémentation enum incomplète** : oublier le point-virgule après `INSTANCE;` quand l'enum possède d'autres membres, ou tenter de définir un constructeur `public` pour une enum à une seule constante (le constructeur d'une enum est toujours implicitement privé, et ne devrait jamais être déclaré `public`).
4. **Tri instable mal géré** : utiliser `>` au lieu de `>=` dans la boucle d'insertion triée changerait l'ordre relatif des scores égaux ; ce n'était pas demandé ici mais c'est le genre de détail qui se teste facilement en relisant son propre code avec un exemple à la main.